

Applied Deep Learning | Hochschule Hof | Prof. Dr. Lars Ackermann-Igl

EFFICIENT LEARNING

Complete Exam-Ready Study Guide — FINAL VERSION

- ◆ Code Analysis: ReduceLROnPlateau + CosineDecayRestarts — parameter by parameter
 - ◆ All Formulas Dissected: Batch Norm, L1 Regularisation
- ◆ Concept Questions: Overfitting, Dropout, Early Stopping
 - ◆ Hardware: CUDA vs TPU explained
 - ◆ Distillation + Quantization with exam-style Q&A
- ◆ Mock-exam style: code explain + formula explain + concept explain

01|LR
Schedulers

02|Normaliza
tion

03|Regulariz
ation

04|Hardware

05|Distillatio
n

06|Quantizati
on

07|Full Q&A;

SECTION 1 — LEARNING RATE SCHEDULERS

Core Concept

Previously we assumed the learning rate (LR) remains **constant** throughout training. Learning rate schedulers **dynamically adjust the LR** during training.

Goal: Improve convergence, escape local minima, and ensure better generalisation. A rate too high → overshooting; too low → slow convergence / stuck in local minimum.

The Four Types — Names, Shapes, and Purposes

Scheduler	How LR changes over time	Best Use
Step Decay	Reduces by fixed factor every n epochs. Looks like a staircase descending.	Simple tasks. When you know roughly when to decay.
Cosine Annealing	Smooth cosine curve from eta_max down to eta_min over T_max steps. With warm restarts: resets to max periodically.	Large-scale training. Standard for Transformers + CNNs.
Cyclic (CLR)	Oscillates between eta_min and eta_max in a triangular/sinusoidal wave. T_0 = cycle length.	Forces exploration. Good when training is unstable.
Warm-Up	Starts at very small LR, ramps up linearly to target LR over first N epochs, then may decay.	Always used with Transformers. Prevents early gradient instability.

CODE ANALYSIS 1: ReduceLROnPlateau (Step Decay variant)

This Keras callback monitors a metric and reduces the LR when improvement stagnates. The professor can show this code and ask: *What does each parameter do?*

```
reduce_lr =  
keras.callbacks.ReduceLROnPlateau(  
  
    monitor="val_loss",  
  
    factor=0.2,  
  
    patience=2,  
  
    verbose=1,  
  
    min_lr=1e-5  
)  
history = train(callbacks=[reduce_lr])
```

- metric observed for stagnation (e.g. validation loss)
- multiply LR by this factor when reducing (new LR = old LR * 0.2)
- wait this many epochs without improvement before reducing LR
- print a message when LR is reduced (logging)
- lower bound – LR never goes below this value
- pass scheduler as callback to training

Parameter-by-Parameter Explanation:

Parameter	What It Does	Effect / Range
monitor	The metric to watch for stagnation. Usually 'val_loss' (validation loss). If this metric stops improving, the LR is reduced.	String. Typically 'val_loss' or 'val_accuracy'
factor	Multiplicative factor applied to the current LR when a reduction is triggered. $\text{new_LR} = \text{old_LR} \times \text{factor}$.	$0 < \text{factor} < 1$. E.g. 0.2 means LR becomes 20% of old value
patience	Number of epochs with no improvement that must pass before the LR is reduced. Acts as a buffer against noisy validation metrics.	Integer. E.g. patience=2 means wait 2 bad epochs then reduce
verbose	Controls logging. If 1, prints a message each time LR is reduced. If 0, silent.	0 = silent, 1 = print message
min_lr	Hard lower bound on the LR. Even if the scheduler would reduce further, LR will not go below this value. Prevents LR from becoming effectively zero.	Float. E.g. $1e-5$ (0.00001)

Example walk-through: factor=0.2, patience=2, starting LR=0.001

→ Epoch 1: val_loss=0.50 (improving) → no change

→ Epoch 2: val_loss=0.51 (no improvement, count=1)

→ Epoch 3: val_loss=0.52 (no improvement, count=2 = patience) → REDUCE LR

→ New LR = $0.001 \times 0.2 = 0.0002$

CODE ANALYSIS 2: CosineDecayRestarts (Cosine Annealing)

```

cosine = keras.optimizers.schedules.CosineDecayRestarts(
    initial_learning_rate=INITIAL_LR,
    first_decay_steps=5*(len(x_train)//128),
    t_mul=2.0,
    m_mul=1.0,
    alpha=0.001
)
history = train(scheduler=cosine)

```

- starting (maximum) LR at the beginning of each cycle
- how many steps the FIRST cosine cycle spans
- each new cycle is t_mul times LONGER than the previous
- scales peak LR down after each restart (1.0 = no reduction)
- minimum LR as fraction of initial_learning_rate
- pass schedule to optimizer

Parameter-by-Parameter Explanation:

Parameter	What It Does	Effect / Range
initial_learning_rate	The maximum (peak) LR at the start of each cosine cycle. After each warm restart, the schedule resets to this peak (modified by m_mul).	Float. E.g. 0.001
first_decay_steps	Number of training STEPS (not epochs) that the first cosine cycle spans before the first restart. Typically calculated as $\text{epochs} \times \text{batches_per_epoch}$.	Integer. E.g. $5 \times (\text{len}(x_train) // 128) = 5$ epochs $\times$ batches

t_mul	After each restart, the NEXT cycle is t_mul times longer than the current one. t_mul=2.0 means each cycle doubles in length: cycle 1 = N steps, cycle 2 = 2N steps, cycle 3 = 4N steps.	Float. 1.0 = all cycles same length; 2.0 = doubling cycles
m_mul	Scales the PEAK learning rate down after each restart. m_mul=1.0 means peak stays the same. m_mul=0.9 means each restart starts at 90% of the previous peak.	Float. 1.0 = same peak every restart; <1.0 = decreasing peaks
alpha	The MINIMUM LR as a fraction of initial_learning_rate. At the bottom of each cosine curve, $LR = \alpha \times \text{initial_learning_rate}$.	Float. E.g. 0.001 means min LR = $0.001 \times \text{initial_LR}$

Cosine Annealing intuition: The LR follows a cosine wave — starts high (initial_learning_rate), smoothly decreases to its minimum ($\alpha \times \text{initial_learning_rate}$), then RESTARTS (warm restart) back to the peak. Each subsequent cycle is t_mul times longer. The periodic restarts help the optimizer escape local minima by briefly exploring with a higher LR.

EXAM TIP: Code questions will ask you to explain each parameter. Key things to memorise: factor = how much LR shrinks; patience = how many bad epochs before shrinking; t_mul = cycle length multiplier; m_mul = peak LR multiplier; alpha = minimum LR fraction.

SECTION 2 — NORMALISATION (BATCH NORM + LAYER NORM)

Why Normalisation? (1 sentence answer)

Problem: Unevenly scaled inputs create an elongated (canyon-shaped) loss surface where large-scale features dominate gradient updates and small-scale features barely contribute — causing slow, zigzagging optimisation.

Fix: Normalise to make the loss surface rounder so the optimizer can move efficiently toward the minimum in all directions.

Batch Normalisation — The 4-Step Formula

Step 1: $\mu = (1/M) * \text{SUM}(A_i)$ Step 2: $\sigma = \text{SQRT}((1/M) * \text{SUM}(A_i - \mu)^2)$

Step 3: $A_{\text{hat}} = (A - \mu) / \sigma$ Step 4: $\text{BN} = \gamma * A_{\text{hat}} + \beta$

Every Symbol in the Batch Norm Formula — Explained:

M	Mini-batch size Number of samples in the current training batch used to compute statistics.
A_i	Activation values The output values from the previous layer for each sample i in the mini-batch.
mu	Batch mean Average activation value across the mini-batch for a given feature. Re-centres data at 0.
sigma	Batch std dev Standard deviation of activations across the mini-batch. Measures spread.
A_{hat}	Normalised activation Activation after re-centring (subtract mu) and rescaling (divide by sigma). Has mean=0, var=1.
gamma	Scale parameter (learnable) Scales the normalised value. LEARNED during training. Allows the network to adjust the variance to whatever the next layer needs.
beta	Shift parameter (learnable) Shifts the normalised value. LEARNED during training. Allows the network to move the mean to whatever the next layer needs.
BN	Batch Norm output Final output of the Batch Norm layer. Has learned mean (beta) and learned variance (gamma). This is passed to the next layer.

Why beta and gamma? After normalising to zero mean / unit variance, the network might actually *need* a different distribution for optimal performance. Beta and gamma are learnable — the network can undo the normalisation if needed. If $\gamma = \sigma_{\text{original}}$ and $\beta = \mu_{\text{original}}$, BN effectively becomes an identity operation. This means BN is never harmful — at worst it learns to do nothing.

Batch Norm — Inference vs. Training (IMPORTANT!)

Phase	Training	Inference
Mean used	Computed from current mini-batch: $\mu = (1/M) * \sum(A_i)$	Uses stored moving average: μ_{mov} (accumulated during training)
Variance used	Computed from current mini-batch: $\sigma^2 = (1/M) * \sum(A_i - \mu)^2$	Uses stored moving average: σ_{mov} (accumulated during training)
Why different?	Many samples in a batch → reliable statistics	Only 1 sample at inference → can't compute batch statistics!
Moving averages	Updated each batch: $\mu_{\text{mov}} = \alpha * \mu_{\text{mov}} + (1 - \alpha) * \mu_{\text{batch}}$	Fixed (used as-is). They represent the 'global' mean/variance.

Batch Norm in Keras Code:

```
model = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(256, use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dense(128, use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dense(10, activation='softmax')
])
```

- disable bias — BN will handle mean shifting via beta
- normalise output BEFORE activation (variant a)
- activation applied AFTER BN
- output layer — no BN needed here

use_bias=False: When using BN, the Dense layer's bias is redundant because BN's beta parameter already shifts the mean. Setting use_bias=False saves parameters and avoids redundancy.

Placement debate: BN can go BEFORE activation (original paper, applied to z) or AFTER activation (applied to a). Both are used — the lecture uses BEFORE activation.

Layer Normalisation vs. Batch Normalisation

Aspect	Batch Normalisation	Layer Normalisation
Normalises ACROSS	All samples in batch, per feature	All features, per single sample
Depends on batch size	YES — needs batch size > 1	NO — works with batch size = 1
Training ≠ Inference	YES — uses moving avgs at inference	NO — identical in both phases

Handles covariate shift	Across batch dimension	Across feature dimension (per-instance)
Best for	CNNs, fully-connected layers (vision)	RNNs, Transformers, NLP tasks

EXAM TIP: Batch Norm = across the BATCH per feature. Layer Norm = across all FEATURES per sample. Batch Norm needs moving averages at inference. Layer Norm does not.

SECTION 3 — GENERALISATION STRATEGIES (REGULARISATION)

The Overfitting Problem

A complex neural network can **memorise training data** — learning noise and specific patterns that don't generalise. Result: **low training error, high test error** = overfitting.

Solution: Change the learning objective to minimise BOTH error AND complexity:

New Loss = Training Error + Complexity Penalty

L1 Regularisation — Formula Breakdown

$$\text{Loss} = \text{Error}(y, \hat{y}) + \alpha * \text{SUM } |w_i|$$

Error(y, y_hat)

Original loss

The standard loss before regularisation (e.g. cross-entropy, MSE). Measures prediction quality.

alpha

Regularisation strength

Scaling factor. Controls how expensive model complexity is. Small alpha = weak penalty. Large alpha = strong penalty, many weights $\rightarrow 0$.

SUM |w_i|

L1 penalty term

Sum of absolute values of ALL weights in the network. This grows with the number and magnitude of non-zero weights.

Full Loss

Penalised objective

The optimizer now minimises both prediction error AND total weight magnitude. Forces the network to use as few/small weights as possible.

Weight Update Rule with L1:

$$w_{\text{new}} = w - \lambda * (d\text{Error}/dw) - \lambda * (d(\alpha * \text{SUM}|w|)/dw) \text{ where } d|w|/dw = +\alpha (w > 0) \text{ or } -\alpha (w < 0)$$

Effect: At every training step, each weight is pushed toward zero by alpha (regardless of gradient). This causes some weights to become *exactly* zero — effectively removing connections. L1 performs implicit **feature selection**.

L2 comparison: L2 uses $\alpha * \text{SUM}(w^2)$ instead. The derivative is $2 * \alpha * w$ — proportional to the weight itself. Large weights shrink fast; small weights shrink slowly. L2 never reaches exactly 0, but keeps all weights small.

Dropout

During training: At each forward pass, randomly deactivate (set to 0) each neuron in the dropout layer with probability p (the dropout rate). Deactivated neurons receive no gradient update during backprop. A *different random subset* is dropped each batch.

Why it works: The network cannot co-adapt neurons (rely on specific combinations). It must learn **redundant, distributed representations**. Equivalent to training an ensemble of $\sim 2^N$ different sub-networks and averaging their predictions.

During inference: All neurons are ACTIVE. Outputs are scaled by $(1-p)$ to keep the expected output magnitude consistent with training.

Example: $p=0.5 \rightarrow$ each neuron has 50% chance of being dropped each batch. At inference, multiply all outputs by 0.5 to compensate.

Early Stopping

Monitor validation loss during training. Stop when it stops improving (for 'patience' epochs). Save the model checkpoint at the best epoch.

Why: After the 'early stopping epoch', training loss keeps decreasing but validation loss increases — the model has started memorising training data.

Overfitting signal: Training accuracy $\gg$ Test accuracy. Training loss $\ll$ Test loss.

All Regularisation Methods — Comparison Table

Method	Category	Core Mechanism	Key Effect
L1	Penalising extrema	Adds $\alpha \cdot \sum w $ to loss	Weights $\rightarrow$ exactly 0. Feature selection.
L2	Penalising extrema	Adds $\alpha \cdot \sum (w^2)$ to loss	Weights small but non-zero.
Dropout	Changing architecture	Random neuron deactivation per batch	Redundant features. Ensemble-like.
Early Stopping	Changing training	Stop at best validation loss	No overfitting past best epoch.
Label Smoothing	Changing training	Replace hard 0/1 labels with soft 0.1/0.9	Prevents overconfident predictions.

EXAM TIP: Dropout at INFERENCE: all neurons active, outputs $\times (1-p)$. L1 pushes weights to 0 (feature selection). L2 keeps weights small but nonzero.

SECTION 4 — HARDWARE ACCELERATION (CUDA & TPUs)

The Bottleneck: Why Normal CPUs are Too Slow

Neural network training = billions of matrix multiplications ($Y = W \cdot X$) every forward+backward pass. A CPU processes these **sequentially** — one operation at a time. A model with 100M parameters trained for 100 epochs needs approximately 10^{18} floating-point operations. On a CPU this could take years. Specialised hardware is essential.

CUDA — Compute Unified Device Architecture

Full name: Compute Unified Device Architecture (by NVIDIA).

What it is: A parallel computing platform + programming API that allows software to run thousands of operations **simultaneously** on GPU cores.

Key principle — SIMD: Single Instruction, Multiple Data. The GPU executes the SAME operation (e.g., multiply-add) on THOUSANDS of different data elements at the same time. Matrix multiplication decomposes perfectly into this pattern.

Framework integration: You never write CUDA code directly. TensorFlow and PyTorch automatically use CUDA when a GPU is detected. Keras uses CUDA via TensorFlow/cuDNN.

In code: Check for GPU: `tf.config.list_physical_devices('GPU')`
Force device: `with tf.device('/GPU:0'): model.fit(...)`

TPUs — Tensor Processing Units

What it is: A custom chip designed by Google *specifically* for neural network matrix operations. Unlike GPUs (general graphics cards adapted for ML), TPUs are purpose-built.

4 Key Strengths:

1. **High Throughput for Matrix Ops:** TPU's systolic array architecture is optimised for large matrix multiplications — the dominant operation in neural networks.
2. **Massive Parallelism:** Performs the same operation on many data elements simultaneously.
3. **High Memory Bandwidth (HBM):** High-Bandwidth Memory feeds data to compute units extremely fast, eliminating the data bottleneck between RAM and processor.
4. **Energy Efficiency:** More performance per watt than CPUs or GPUs — lower cost, lower carbon footprint for large-scale training.

Result: Faster training + lower cost + ability to train larger models.

Used via: Google Cloud TPUs. Supported by TensorFlow/Keras, JAX, and PyTorch (via XLA).

Hardware	Made by	Core Strength	DL Suitability
CPU	Intel/AMD	Sequential general computing	Poor — too slow for large DL
GPU + CUDA	NVIDIA	Parallel SIMD matrix ops	Good — industry standard
TPU	Google	Purpose-built matrix ops + HBM	Excellent — fastest for DL

EXAM TIP: CUDA = Compute Unified Device Architecture (NVIDIA software for GPU parallelism). TPU = Google's custom chip. HBM = High Bandwidth Memory. SIMD = Single Instruction Multiple Data.

SECTION 5 — MODEL DISTILLATION

Core Concept: Teacher → Student

Problem: Large models (GPT, ResNet-152) are accurate but too slow/large for deployment on mobile, edge devices, or real-time systems.

Model distillation (= knowledge distillation) transfers the learned knowledge from a large **teacher model** to a small, efficient **student model**. The student learns to mimic the teacher's behaviour, achieving near-teacher accuracy at a fraction of the compute cost.

Why Soft Labels? (The Key Insight)

Hard labels (standard training): [0, 0, 1, 0, 0] — just tells the model which class is correct. Provides minimal information.

Soft labels (teacher outputs): [0.02, 0.05, 0.70, 0.20, 0.03] — the teacher's full probability distribution. This is richer: it tells the student not just which class is correct, but also that class 3 and class 4 look similar (both have non-trivial probabilities). The student learns **inter-class relationships** from these soft labels, not just the correct class.

Distillation Variants:

Variant	Description
Soft Target Distillation	Student trained to match teacher's output logits/probabilities using KL divergence. Most common form.
Cross-Modal Distillation	Teacher and student work in different modalities (e.g., teacher=images, student=text). Transfers knowledge across data types.
GAN-based Distillation	Discriminator tries to tell teacher from student outputs apart. Student learns to be indistinguishable from teacher.

SECTION 6 — QUANTIZATION

The Problem: FP32 is Wasteful

Neural network weights are stored as **FP32** (32-bit floating point, range $\pm 3.4 \times 10^{38}$, 7 decimal places). For a model with 1 billion parameters: $1\text{B} \times 4 \text{ bytes} = 4 \text{ GB just for weights}$.

Key observation: In practice, trained weights cluster in a tiny fraction of the FP32 range (e.g., -2 to +2). The vast majority of the FP32 precision is wasted.

What Quantization Does:

Quantization: $\text{map FP32 } [f_min, f_max] \rightarrow \text{INT8 } [-128, 127]$ | Memory saving: 4 bytes $\rightarrow$ 1 byte = 4x

Type	When Applied	Accuracy	Effort
PTQ Post-Training Quantization	After training is complete. Static, one-time conversion.	Some accuracy loss — model was not trained for lower precision.	Easy — apply to any pre-trained model.
QAT Quantization-Aware Training	During training — simulate quantization effects. Model adapts to lower precision.	Better accuracy — model explicitly compensates for quantization errors.	Requires re-training or calibration step.

EXAM TIP: FP32→INT8 = 4x memory saving. PTQ = simple but accuracy loss. QAT = better accuracy needs retraining.

SECTION 7 — EXAM Q&A; (Mock-Exam Style Questions + Full Answers)

Exam question types from mock paper: (A) Code analysis — explain what each parameter does. (B) Formula breakdown — explain each symbol. (C) Fill table / compare two approaches. (D) Concept questions — explain in 2–4 sentences. (E) Identify errors / give correction.

EXAM QUESTION Q1 (Code Analysis) [~4 Points]

The following Keras code implements a learning rate scheduler. Explain what each highlighted parameter does:

```
reduce_lr = keras.callbacks.ReduceLROnPlateau(  
    monitor='val_loss', factor=0.2, patience=2, min_lr=1e-5  
)
```

✓ MODEL ANSWER

monitor='val_loss': Specifies which metric to observe for stagnation. Here, validation loss is monitored. When this metric stops improving, the scheduler triggers a learning rate reduction.

factor=0.2: The multiplicative factor applied to the current learning rate when a reduction is triggered. The new learning rate = old LR $\times$ 0.2. For example, if LR = 0.001, after reduction LR = 0.0002.

patience=2: The number of consecutive epochs with no improvement in the monitored metric that must pass before the LR is reduced. Here, 2 epochs without improvement triggers the reduction. This prevents reacting to random fluctuations.

min_lr=1e-5: A hard lower bound on the learning rate. Even if the scheduler would reduce further, the LR will never go below 0.00001. Prevents the LR from becoming so small that training effectively stops.

EXAM QUESTION Q2 (Code Analysis) [~4 Points]

Explain the parameters `t_mul` and `m_mul` in the `CosineDecayRestarts` scheduler. What would happen if `t_mul=1.0` instead of `t_mul=2.0`?

✓ MODEL ANSWER

t_mul=2.0: After each warm restart, the next cycle is `t_mul` times LONGER than the previous cycle. With `t_mul=2.0`: if the first cycle spans 100 steps, the second spans 200 steps, the third spans 400 steps, and so on. This allows increasingly fine-grained exploration as training progresses.

m_mul=1.0: Scales the peak learning rate at each restart. `m_mul=1.0` means the peak LR is the same at every restart (no decay of the maximum). If `m_mul=0.9`, each restart begins at 90% of the previous peak, gradually reducing the overall maximum LR over time.

If t_mul=1.0: All cycles would have the same length. There would be no progressive lengthening. The schedule would restart at the same peak LR every fixed number of steps — like a purely periodic (non-progressive) annealing schedule.

EXAM QUESTION Q3 (Formula) [~4 Points]

The Batch Normalization output is given by: $BN = \gamma * \hat{A} + \beta$, where $\hat{A} = (A - \mu) / \sigma$. Explain the roles of γ and β . Why are they necessary despite the normalisation already being applied?

✓ **MODEL ANSWER**

gamma (scale): A learnable parameter that scales the variance of the normalised activation. It controls how spread out the normalised values are. Learned during training via backpropagation.

beta (shift): A learnable parameter that shifts the mean of the normalised activation. It controls the centre of the distribution after normalisation. Learned during training via backpropagation.

Why they are necessary: After normalisation, all activations have mean=0 and variance=1. But this might not be the optimal distribution for the next layer. For example, if the next layer is ReLU, having the mean at 0 means roughly half the outputs will be zeroed. γ and β allow the network to RE-LEARN whatever distribution is optimal. If $\gamma = \sigma_{original}$ and $\beta = \mu_{original}$, BN becomes an identity — it can undo itself completely. This ensures BN is never harmful: at worst it learns to do nothing.

EXAM QUESTION Q4 (Concept) [~3 Points]

Why does Batch Normalisation behave differently during training and during inference? What are moving averages and why are they needed?

✓ **MODEL ANSWER**

During training: Batch Norm computes mean and variance from the current mini-batch. This is fine because there are M samples to compute reliable statistics from.

During inference: We typically process one sample at a time — there is no 'batch' to compute statistics from. We cannot compute a meaningful mean and variance from a single sample.

Moving averages: During training, Batch Norm maintains exponentially weighted moving averages of the batch means and variances: $\mu_{mov} = \alpha * \mu_{mov} + (1 - \alpha) * \mu_{batch}$. Over many batches, these averages converge to the true population mean and variance. At inference, these stored moving averages are used instead of batch statistics, making inference possible for single samples.

EXAM QUESTION Q5 (Fill Table) [~3 Points]

Compare the training error and test error for an overfitting model and a properly regularised model. What does each regularisation technique add to the loss function?

✓ **MODEL ANSWER**

Overfitting model: Training error = LOW (near zero). Test error = HIGH. Large gap between them.

Properly regularised model: Training error = moderate. Test error = similar to training error (small gap).

L1 adds: $\alpha * \sum |w_i|$ to the loss. Penalty = sum of absolute weight values.

L2 adds: $\alpha * \sum (w_i^2)$ to the loss. Penalty = sum of squared weight values.

Dropout adds: No explicit term to loss — instead structurally changes the network by randomly deactivating neurons, forcing redundant representations.

EXAM QUESTION Q6 (Compare) [~3 Points]

What is the key structural difference between Batch Normalisation and Layer Normalisation? For which type of model is each preferred, and why?

✓ MODEL ANSWER

Batch Normalisation: Normalises ACROSS the batch dimension — computes mean and variance for each feature across all M samples in the mini-batch. Statistics depend on batch size. Requires moving averages for inference. Preferred for CNNs and fully-connected networks (computer vision) where features from different images are comparable.

Layer Normalisation: Normalises ACROSS all features for each individual sample independently. Statistics are computed within one sample — completely independent of batch size. Same computation at training and inference. Preferred for RNNs and Transformers (NLP/sequential tasks) where batch size is often small or variable, and the batch dimension is not meaningful.

EXAM QUESTION Q7 (Concept) [~3 Points]

What is knowledge distillation? Explain the teacher-student concept and why soft labels are more informative than hard labels.

✓ MODEL ANSWER

Knowledge distillation transfers learned knowledge from a large accurate model (teacher) to a smaller efficient model (student). The student does not learn only from the original training data — it also learns to mimic the teacher's full output distribution.

Hard labels = one-hot vectors (e.g., [0,0,1,0]). They only tell the model which class is correct — binary information with no nuance.

Soft labels = teacher's probability distribution (e.g., [0.02, 0.03, 0.70, 0.20, 0.05]). These carry rich information: the probabilities for non-correct classes reveal class similarities and relationships (e.g., class 3 and class 4 are similar). The student learns these relationships in addition to the correct classification — resulting in better generalisation with a smaller model.

EXAM QUESTION Q8 (Compare) [~3 Points]

Explain the difference between Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT). What memory saving does INT8 provide over FP32?

✓ MODEL ANSWER

Memory saving: FP32 = 32 bits = 4 bytes per value. INT8 = 8 bits = 1 byte per value. Saving = 4x reduction in memory. A 4 GB model becomes 1 GB.

PTQ — Post-Training Quantization: The model is first trained normally in FP32. After training is complete, weights are quantized to INT8. Simple and fast to apply to any pre-trained model. Drawback: accuracy loss because the model was never exposed to lower precision during training — it doesn't know how to compensate.

QAT — Quantization-Aware Training: Quantization effects are simulated during the training process. The model experiences lower precision while gradients are still available to adapt the weights. The model explicitly learns to compensate for quantization errors. Result: better accuracy than PTQ. Requires re-training or a calibration training step.

FINAL CHEAT SHEET — EFFICIENT LEARNING

All Key Formulas

Batch Norm: $A_hat = (A - \mu)/\sigma$ → $BN = \gamma * A_hat + \beta$

L1 Loss = Error + $\alpha * \sum |w|$ | **L2 Loss** = Error + $\alpha * \sum (w^2)$

FP32(4 bytes) → INT8(1 byte) = 4x memory saving | **Dropout inference:** output × (1-p)

Topic	Key Point 1	Key Point 2	Exam Gotcha
ReduceLROnPlateau	factor=how much LR shrinks	patience=how many bad epochs before LR shrink	one LR hard lower bound
CosineDecayRestarts	t_mul = cycle length multiplier	m_mul = peak LR multiplier after restart	alpha = min LR fraction of initial
Batch Norm	Normalise across batch per feature	γ =scale, β =shift (learnable)	Inference uses MOVING AVERAGES, not batch
Layer Norm	Normalise across features per sample	Works with batch size = 1	Same at train and inference (no moving avgs ne
L1 Regularisation	Adds $\alpha * \sum w $ to loss	Pushes weights to EXACTLY 0	Feature selection — some weights become 0
L2 Regularisation	Adds $\alpha * \sum (w^2)$ to loss	Keeps weights small but NON-ZERO	Default regulariser
Dropout	Randomly drop neurons each batch	Deactivated neurons get NO gradient	At INFERENCE: all active, multiply by (1-p)
Early Stopping	Stop when val_loss stops improving	Save best model checkpoint	Use patience parameter
CUDA	NVIDIA parallel computing platform	SIMD: same op on many data at once	TF/PyTorch use it automatically
TPU	Google custom chip for matrix ops	HBM = High Bandwidth Memory	4x faster than GPU for DL training
Distillation	Teacher (large) → Student (small)	Soft labels richer than hard labels	Student learns inter-class relationships
PTQ	Quantize AFTER training	Simple but some accuracy loss	FP32→INT8 = 4x memory saving
QAT	Simulate quantization DURING training	Model adapts to lower precision	Better accuracy than PTQ, needs retraining

EXAM STRATEGY — Efficient Learning (6 rules):

- Code questions:** For every parameter, state: (a) what it does, (b) what happens if you increase/decrease it.
- Batch Norm formulas:** Know all 4 steps: compute μ → compute σ → normalise → scale+shift.
- BN vs LN:** 'Across batch' vs 'across features'. BN = needs moving avgs at inference. LN = does not.
- Regularisation:** Always mention training error vs test error. L1→weights=0. L2→weights small. Dropout→scale at inference.
- Hardware:** CUDA = NVIDIA software. TPU = Google custom chip. HBM = high bandwidth memory.
- Quantization:** Always state the 4x memory saving (FP32→INT8). PTQ=simple. QAT=accurate.